Algorithmic Problem Solving 2026

Project Report

Group	Group H
Members	Karl Thedor Ruby (krub@itu.dk) Kristoffer Mejborn Eliasson (krme@itu.dk) Lukas Shaghashvili-Johannessen (lush@itu.dk)
Supervisor	Holger Dell, Martin Aumüller
Course	Algorithmic Problem Solving (APS 2026)
Date	[YYYY-MM-DD]

IT University of Copenhagen

Contents

1. Abstract

Provide a short summary (max 200 words) describing the designed problem, the solved problems, and the main algorithmic techniques used.

2. Designed Problem

Provide a self-contained problem that can be used for automated judging. Make sure `verifyproblem` accepts your files.

2.1. Problem description

Give the problem statement as you would present it to contestants. Include input and output formats, constraints, and examples.

2.2. Intended solution

Describe the algorithmic idea behind an intended (accepted) solution. Use pseudocode and a short complexity analysis (time and memory). Example:

```
function solve(input):
  parse input
  // algorithmic steps
  return output
```

- **Time complexity:** O(…)
- **Memory:** O(…)

2.3. Test-case design

Explain how you generated sample and secret inputs. For each test group, state the purpose, parameter ranges, and expected corner cases.

- **Sample tests:** small cases for manual checking.
- **Secret tests:** randomized and worst-case instances to exercise complexity.

2.4. Accepted and wrong solutions

Include at least one accepted solution and one or more wrong/inefficient variants. For each wrong solution, explain why it fails (e.g., incorrect logic, TLE, memory).

2.5. Input validation & formatting

Specify how your judge should treat malformed input and which assumptions contestants may rely on.

2.6. Verify notes

Confirm that `verifyproblem` runs cleanly. If it produced warnings, document them and the fixes applied.

2.7. Testing and Validation

- Describe your testing strategy
- Provide commands to reproduce tests.

Example commands:

2.8. Source for Designed Problem

Provide a compact description of the repository layout and how to run the judge and tests locally.

- **Files to include in zip:** problem statement, solutions, test generator(s), verifyproblem configuration, README.
- **Run locally:** provide exact commands used to generate and run tests.

3. Solved Problems

3.1. Robert Hood (Geometric Algorithms)

3.1.1. Problem metadata

- **Problem name / Kattis link:** Robert Hood
- **Short formal I/O description:** Given n points in the plane with integer coordinates, output the maximum Euclidean distance between any two points, as a floating-point number.

3.1.2. Solution overview

The maximum distance between any two points in a finite set is always achieved by two points on the convex hull of the set. This reduces the problem to two steps: compute the convex hull, then find the diameter of the hull polygon.

The convex hull is computed using Andrew's monotone chain algorithm, which sorts the points lexicographically and constructs the lower and upper hull in two linear passes. The result is the vertices of the hull in counter-clockwise order.

The diameter of the convex polygon is then found using the rotating calipers technique. For each edge of the hull, the algorithm advances an antipodal pointer j as long as the next vertex is farther from the edge than the current one. The cross product of the edge direction and the vector between consecutive candidate points determines the direction to move j. Both endpoints of the current edge are checked against j at each step, so no antipodal pair is missed.

This approach was chosen because it achieves optimal asymptotic complexity and avoids the $O(m^2)$ brute-force over hull vertices, which could be slow when all input points lie on the hull.

3.1.3. Correctness argument

Reduction to hull. Any point strictly inside the convex hull cannot be a diameter endpoint, since projecting it outward to the hull boundary only increases distance. Therefore the maximum distance is realised by two hull vertices.

Rotating calipers. The algorithm maintains the invariant that j is the index of the vertex farthest from the directed edge between `hull[i]` and `hull[i+1]`, among all indices not yet "passed" by i. The cross product check `cx > 0` advances j when the next vertex is strictly farther, and stops otherwise. Because the hull is convex, the distance from an edge is unimodal over consecutive vertices, so the pointer j never needs to retreat. Both `hull[i]` and `hull[(i+1) % m]` are compared to `hull[j]` at each step to cover all antipodal pairs correctly.

Edge cases. When $m = 1$ (all points identical) the distance is 0. When $m = 2$ (collinear or just two distinct points) the distance is computed directly. These are handled before the calipers loop.

3.1.4. Complexity and time-limit reasoning

Let n be the number of input points and m the number of hull vertices, with $m \leq n$.

- **Convex hull:** $O(n \log n)$ dominated by the initial sort.
- **Rotating calipers:** $O(m)$ since each pointer advances at most m times.

- **Overall:** $O(n \log n)$ time and $O(n)$ space.

For $n = 10^5$, this is approximately $10^5 \times 17 \approx 1.7 \times 10^6$ operations for the sort, plus a linear pass. The implementation avoids square roots until the final step by comparing squared distances, eliminating floating-point overhead from the inner loop.

3.1.5. Empirical worst-case testing

The worst case for the rotating calipers occurs when all input points lie on the convex hull, maximising m. A set of n points uniformly distributed on a large circle achieves this. The following generator was used:

```
import math
N = 100000
print(N)
for i in range(N):
    angle = 2 * math.pi * i / N
    x = round(1e9 * math.cos(angle))
    y = round(1e9 * math.sin(angle))
    print(x, y)
```

Running this input on the solution completed in approximately 0.73 seconds in CPython 3, which is within the Kattis time limit.

3.1.6. Implementation notes

- **Language used:** Python 3
- **Files:** `solutions/geometry/roberthood.py`
- **How to run:** `python3 solutions/geometry/roberthood.py < input.txt`

3.2. Cookie Selection (Algorithms on Arrays)

3.2.1. Problem metadata

- **Problem name / Kattis link:** Cookie Selection
- **Short formal I/O description:** Given a sequence of up to 2×10^5 events, each either inserting a positive integer into a multiset or querying and removing the element at rank $\lfloor \frac{n}{2} \rfloor + 1$ (1-indexed) from the sorted multiset of current size n, output each queried value on its own line.

3.2.2. Solution overview

The core challenge is maintaining a dynamic multiset that supports two operations in sublinear time: insertion of an element and extraction of the element at a given rank. A naive sorted list achieves $O(n)$ per operation, which is too slow for $n = 2 \times 10^5$.

The solution uses a Fenwick tree over coordinate-compressed values. Because cookie diameters are up to 10^9 but there are at most 2×10^5 distinct values in the input, all values are mapped to contiguous indices $1, 2, \ldots, m$ where m is the number of distinct values. The BIT stores counts: position i holds how many cookies with compressed value i are currently in the holding area. A prefix sum query then gives the number of cookies with value at most i.

To answer a rank query for rank k, the solution uses a binary descent on the BIT structure, finding the smallest index i such that the prefix sum up to i is at least k. This descent runs in $O(\log m)$ rather than $O(\log^2 m)$ compared to binary searching over prefix sum queries.

This approach was selected because the BIT offers simple, cache-friendly implementation while achieving the necessary $O(\log n)$ per operation.

3.2.3. Correctness argument

Coordinate compression. All values are read before processing begins. The mapping compress : $v \mapsto i + 1$ is a bijection from the set of distinct input values to $\{1, \ldots, m\}$, preserving order. Thus rank in the compressed BIT equals rank in the original multiset.

BIT invariant. After each insertion of value v, the BIT count at position compress(v) is incremented by one. After each removal, it is decremented by one. Therefore, at any point, `bit.query(i)` equals the number of cookies currently in the holding area with compressed index at most i, which equals the number of cookies with diameter at most decompress(i).

Rank formula. The problem specifies that for n cookies the cookie sent is at position $\lfloor \frac{n}{2} \rfloor + 1$ in sorted order for both odd and even n. Setting `rank = n // 2 + 1` before each removal matches this specification exactly.

kth correctness. The binary descent in `kth(k)` maintains the invariant that `prefix_sum[1..pos] < k` throughout the loop and terminates with the smallest `pos + 1` satisfying `prefix_sum[1..pos+1] >= k`, which is exactly the k-th occupied position.

3.2.4. Complexity and time-limit reasoning

Let N be the number of input lines and $m \leq N$ the number of distinct cookie diameters.

- **Preprocessing:** sorting the distinct values takes $O(N \log N)$.
- **Per event:** each insertion and each rank query performs one BIT update or descent, each costing $O(\log m) = O(\log N)$.
- **Overall:** $O(N \log N)$ time and $O(N)$ space.

For $N = 2 \times 10^5$ and a typical time limit of 1-2 seconds, roughly $2 \times 10^5 \times 17 \approx 3.4 \times 10^6$ elementary operations are required, which is well within budget even in Python given the constant factor of the BIT operations.

3.2.5. Empirical worst-case testing

A worst-case input alternates insertions and removals to keep the holding area non-empty throughout. The following generator was used:

```
N = 100000
for i in range(1, N + 1):
    print(i)
for _ in range(N):
    print('#')
```

This produces 2×10^5 lines: 10^5 insertions followed by 10^5 queries, exercising both the coordinate compression and the BIT at maximum size. Running this input on the solution completed in approximately 0.67 seconds in CPython 3, which is within the Kattis time limit.

3.2.6. Implementation notes

- **Language used:** Python 3
- **Files:** `solutions/arrays/cookieselection.py`
- **How to run:** `python3 solutions/arrays/cookieselection.py < input.txt`

3.3. Free Real Estate (Randomized Algorithms)

3.3.1. Problem metadata

- **Problem name / Kattis link:** Free Real Estate
- **Short formal I/O description:** Given an array of n integers all initialised to a, process q operations: point updates setting index i to value v, and range sum queries over $[s, e]$. For each query, output the sum or the string "it's free real estate" if the sum equals zero.

3.3.2. Solution overview

The problem requires point updates and range prefix-sum queries over a mutable array. A Fenwick tree (BIT) supports both operations in $O(\log n)$ time and $O(n)$ space, which is optimal for this problem structure.

Rather than storing the actual prices in the BIT, the solution stores only the **delta** from the initial value a. The BIT is initialised to all zeros. When penthouse i is updated to value v, the change `v - vals[i]` is added to the BIT at position i, and `vals[i]` is updated to v. For a range query $[s, e]$, the sum is computed as $(e - s + 1)c \cdot a + \text{BIT.query}(e) - \text{BIT.query}(s - 1)$, where the first term accounts for the initial uniform price and the BIT terms account for all accumulated deltas.

This representation avoids initialising the BIT with n values, keeping setup to $O(1)$ instead of $O(n \log n)$, which matters when n is large.

3.3.3. Correctness argument

Delta invariant. The BIT stores deltas from the initial value a, not absolute prices. After any sequence of updates, `BIT.query(j)` equals the sum of all changes applied to positions 1 through j.

Range sum. For a query $[s, e]$, the total price is the base contribution $(e - s + 1)c \cdot a$ plus the net deltas over that range, which is `BIT.query(e) - BIT.query(s-1)`. This matches the code exactly.

Update correctness. Each update computes `delta = v - vals[i]` before writing the new value to `vals[i]`. This means each delta is always relative to the actual current price, so repeated updates to the same index accumulate correctly in the BIT.

3.3.4. Complexity and time-limit reasoning

Let n be the array length and q the number of operations.

- **Initialisation:** $O(n)$ to allocate `vals`, $O(1)$ BIT setup.
- **Per update:** one BIT point update in $O(\log n)$.
- **Per query:** two BIT prefix queries in $O(\log n)$.
- **Overall:** $O(n + q \log n)$ time, $O(n)$ space.

For the largest test group ($n, q = 10^6$), this requires approximately $2 \times 10^6 \times 20 = 4 \times 10^7$ operations. The implementation uses `sys.stdin.buffer` for fast binary reads, avoiding the overhead of Python's line-by-line input parsing, which is critical at this scale.

3.3.5. Empirical worst-case testing

The worst case combines maximum array size with interleaved updates and queries over the full range. The following generator was used:

```
import random
N = 1000000
Q = 1000000
```

```
A = 1
print(N, Q, A)
for i in range(Q):
    if i % 2 == 0:
        idx = random.randint(1, N)
        v = random.randint(-10**9, 10**9)
        print(f'update {idx} {v}')
    else:
        s = random.randint(1, N)
        e = random.randint(s, min(s + 1000, N))
        print(f'query {s} {e}')
```

Running this input on the Kattis judge completed in approximately 0.48 seconds in CPython 3, well within the time limit.

3.3.6. Implementation notes

- **Language used:** Python 3
- **Files:** `solutions/random/freerealestate.py`
- **How to run:** `python3 solutions/random/freerealestate.py < input.txt`